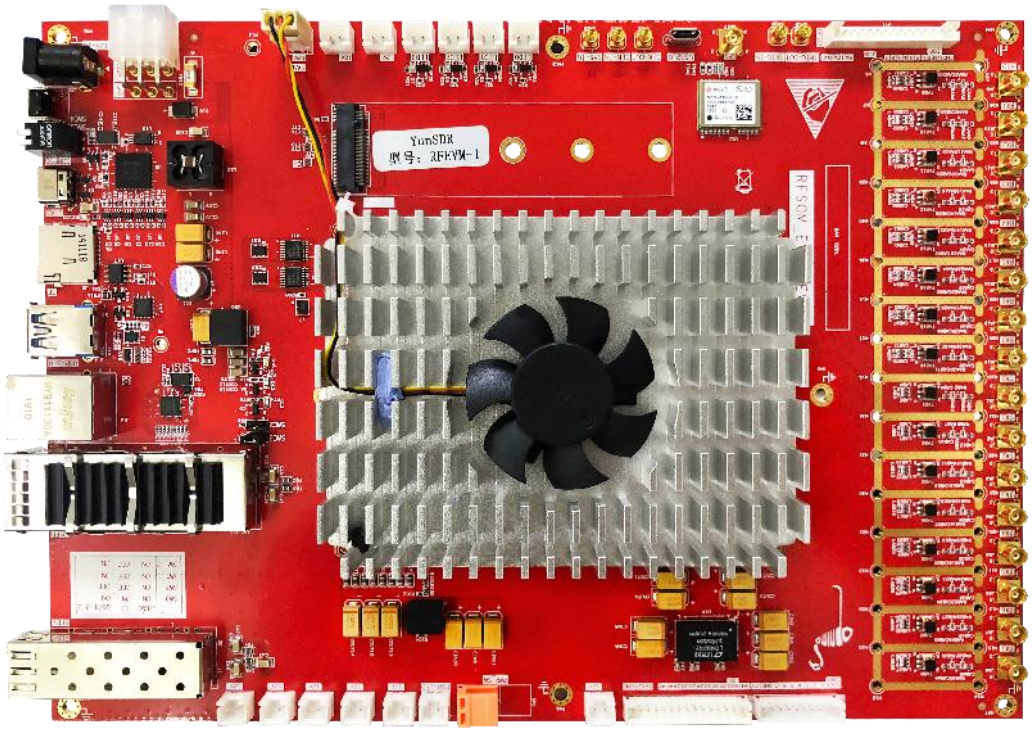
RFEVM 开发平台

RF 开发指南

Rev. 1.0



目录

1 简介 3
2. FPGA 最小系统搭建 3
2.1 硬件架构介绍 4
2.1.1 RFSoC 器件时钟转发特性 4
2.1.2 RFEVM 开发板时钟结构 6
2.2 RF 数据转换器配置指南 7
RF-ADC 片设置 9
RF-DAC 片配置 11
系统时钟配置 14
高级配置 16
2.3 FPGA 硬件工程介绍 18
2.4 裸机应用程序 22
2.5 运行 demo 例程 28
附录 A-API 介绍 34

1 简介

RFEVM 开发板面向开箱即用评估及前沿应用开发的理想 RF 测试平台。开发板包含 Zynq UltraScale+ RFSoC ZU47DR，在一款芯片内集成射频直接采样数据转换器、FPGA 逻辑、完整的 ARM 处理器子系统和高速收发器等。

第三代 RFSoC 器件与前几代产品相比，射频输入输出频率响应已扩展至全面支持 6GHz 以下频段，可帮助用户开发尖端 RF 设计，例如大规模 MIMO 无线电、5G 基带、固定无线接入、测试测量与相控阵雷达等等。

主要特性：

- ZU47DR 射频片上系统（RFSoC）的一体化器件
- 具有内置数字上下变频和插值抽取功能
- 支持 8 路 14 位发送，8 路 14 位接收
- 支持 1MHz~6GHz 频点
- 支持 1x,2x,3x,4x,5x,6x,8x,10x,12x,16x,20x,24x,40x 的插值/抽取系数

2. FPGA 最小系统搭建

本文档主要介绍 RFEVM 的 FPGA 射频开发流程，包括以下几部分：

- RFEVM 射频硬件结构介绍
- RF 数据转换器配置指南
- FPGA 硬件工程介绍
- 裸机配置流程介绍

通过本参考例程用户可以了解本系统硬件构成，编程架构，对于做 FPGA 开发的用户，可以基于这个 FPGA 工程做二次开发，实现空口传输基带数据。

2.1 硬件架构介绍

2.1.1 RFSoC 器件时钟转发特性

RFSoC 在芯片内集成了数据转换器，第三代 RFSoC 系列器件均包含 4 个 ADC Tile 和 4 个 DAC Tile，每个 ADC 和 DAC Tile 内可能包含 1 个（ZU43DR）或 2 个（ZU47DR/48DR）或者 4 个（ZU49DR）Channel，这取决于芯片型号。

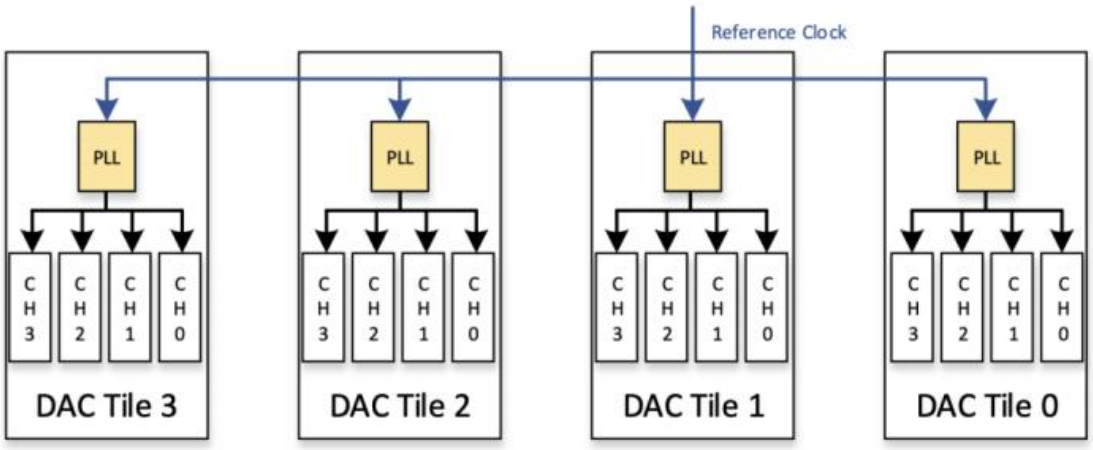
RFEVM 开发板上使用 ZU47DR 器件，每个 ADC Tile 包含 2 路 ADC，每个 DAC Tile 包含 4 路 DAC。

每个 Tile 都可以独立配置，工作在不同的采样时钟频率上，采样时钟可以来自于外部高频时钟输入，可以来自与每个 Tile 内部 PLL 倍频后的时钟，也可以来自于其他 Tile 分发的参考时钟。

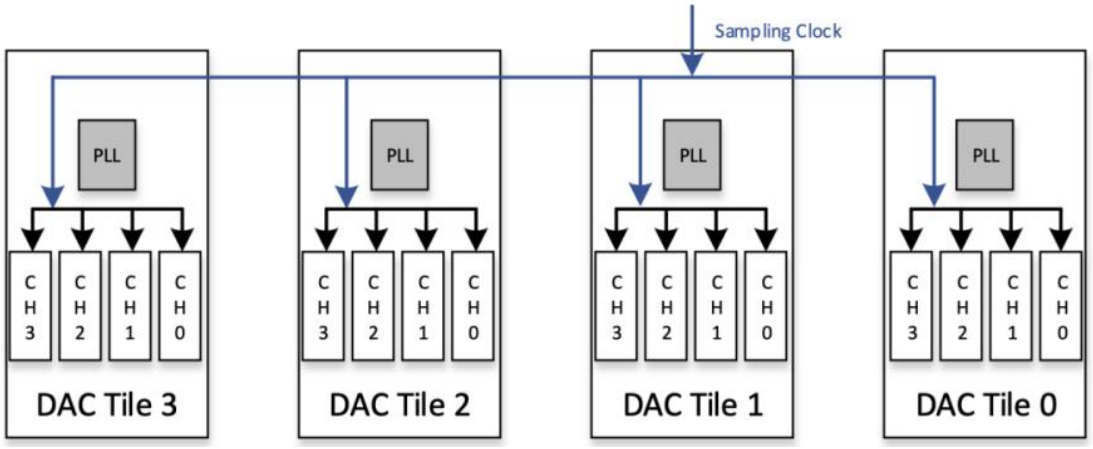
第三代 RFSoC 器件相比之前几代产品引入了时钟转发特性，不在需要为每个 Tile 都提供独立的采样时钟，节省了引脚数量，降低了外围电路的复杂度。

时钟转发的方式共分为两种，一种是参考时钟转发，一种是采样时钟转发。

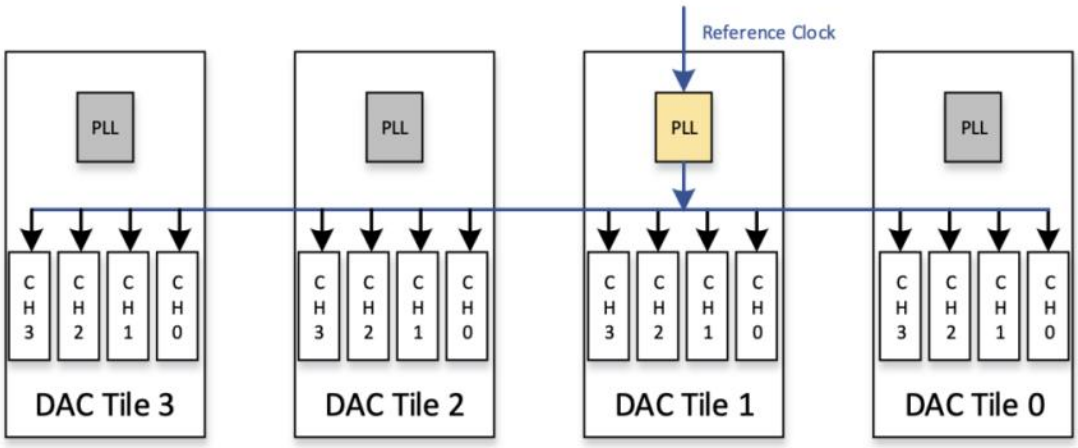
所谓参考时钟转发，是指外部时钟芯片提供低噪声低频时钟到一个 ADC/DAC Tile 的专用时钟管脚，该时钟管脚在 Tile 间转发，通过每个 Tile 内的 PLL 倍频出所需的采样时钟。



而采样时钟转发有两种途径，一是外部时钟芯片直接提供低噪声高频的采样时钟到一个 ADC/DAC Tile 的专用时钟管脚，采样时钟将在 Tile 间分发，无需通过内部 PLL 倍频，直接提供给 ADC/DAC Channel 作为采样时钟；



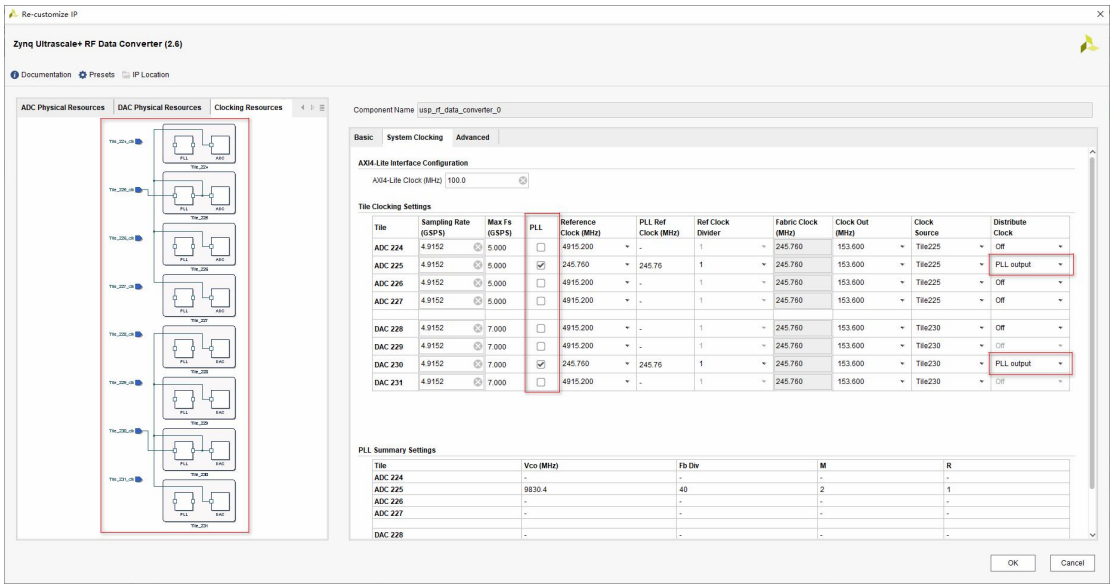
时钟芯片提供低噪声低频时钟到一个 ADC/DAC Tile 的专用时钟管脚，该时钟在此 Tile 内通过 PLL 倍频到采样频率，随后将此采样时钟在 Tile 间分发。



以上仅为时钟分发方式的简化解释，会有一些使用的限制和建议，建议用户参考 PG269 Chapter 4 Clock-on-chip Clock Distribution 章节阅读。

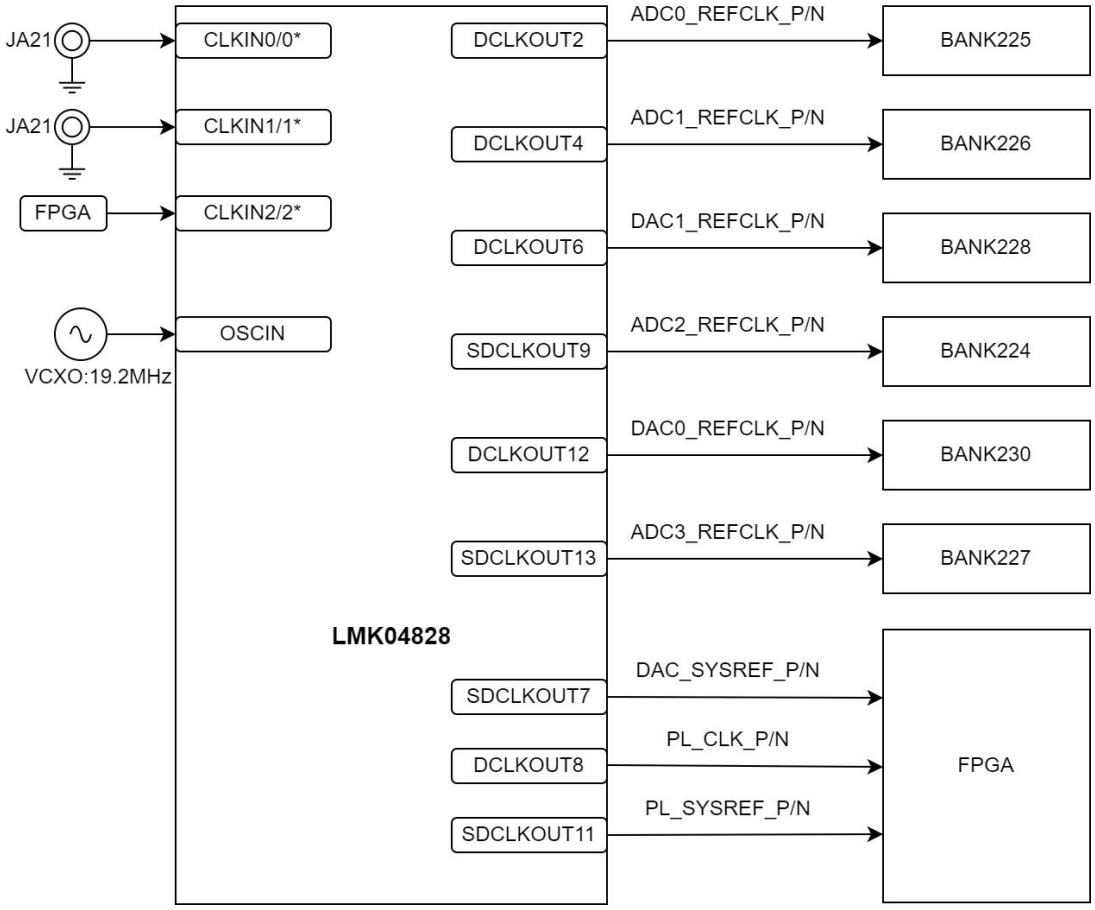
用户也可以打开 Vivado，选择器件型号后在 RF Data Converter IP 内根据原理图和目标设计进行配置。

下图是 RFEVM 开发板时钟时钟结构配置：



2.1.2 RFEVM 开发板时钟结构

这里我们仅介绍 RFEVM 开发板与 RF 数据转换器相关的时钟部分，这部分时钟均由 RFSOM 核心板上 LMK04828 时钟芯片提供。



LMK04828 是一个双环路抖动清除器和时钟发生器，一级回路的参考时钟输入可为外部参考时钟、FPGA 提供的的时钟；二级回路的参考输入为板上的 19.2MHz VCXO。其中 DAC_REFCLK 和 ADC_REFCLK 作为 RF-ADC/RF-DAC 低频参考时钟输入；PL_CLK、PL_SYSREF 和 DAC_SYSREF 均可作为 MTS(Multi-Tile Synchronization)应用。

RFEVM 开发板上的芯片型号为 ZU47DR，其 4 个 ADC Tile 和 4 个 DAC Tile 都有专用的时钟输入引脚，但只有四个 ADC Tile（Bank 224/225/226/227）和两个 DAC Tile（Bank 228/230）的时钟管脚被引出。如下表所示为详细连接关系：

ADC/DAC Tile	信号名	时钟输出源 LMK04828
ADC Tile 224	ADC_REFCLK_N2/P2_R	SDCLKOUT9
ADC Tile 225	ADC_REFCLK_N0/P0_R	DCLKOUT2
ADC Tile 226	ADC_REFCLK_N1/P1_R	DCLKOUT4
ADC Tile 227	ADC_REFCLK_N3/P3_R	SDCLKOUT13
DAC Tile 228	DAC_REFCLK_N1/P1_R	DCLKOUT6
DAC Tile 229	无	
DAC Tile 230	DAC_REFCLK_N0/P0_R	DCLKOUT12
DAC Tile 231	无	

2.2 RF 数据转换器配置指南

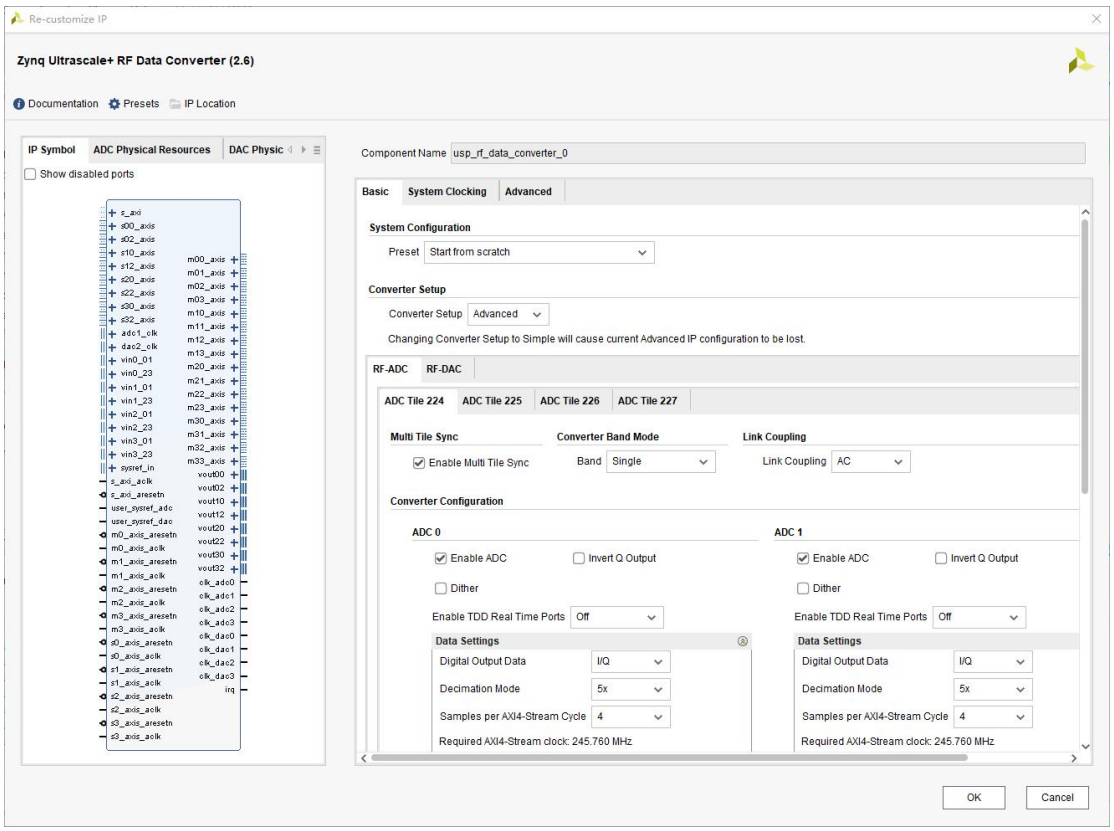
RFSoC 射频数据转换器 IP 核提供了一个可配置的封装器，允许 RF-ADC 和 RF-DAC 块用于 IP 集成器设计。

具体细节可参考官方文档：

https://docs.xilinx.com/r/en-US/pg269-rf-data-converter

这里以 RFEVM（xczu47dr）开发平台为例，去介绍 IP 的相关配置。

IP 基础配置



Basic 页面

Presets（预配置）

Vivado IDE IP 核心配置界面提供了一种保存和应用预设配置的方法。从预设菜单，几个固定配置是可用的。

ADC:R2C Multi x2 DAC:C2R Multi x2(Beta)：RF-ADC 配置在 x2 多波段使用实转虚的数据，RF-DAC 配置在 x2 多波段使用虚转实的数据。

DAC:C2C Multi 2x2(Beta)：在单个 RF-DAC 块上的 2x2 多频段使用虚到虚的数据。

DAC:C2R Multi 2x2(Beta)：在单个 RF-DAC 块上的 2x2 多频段使用虚到实的数据。

DAC:C2R Multi x4(Beta)：在单个 RF-DAC 块上的 1x4 多频段使用虚到实的数据。

Converter Setup（变频器的设置）

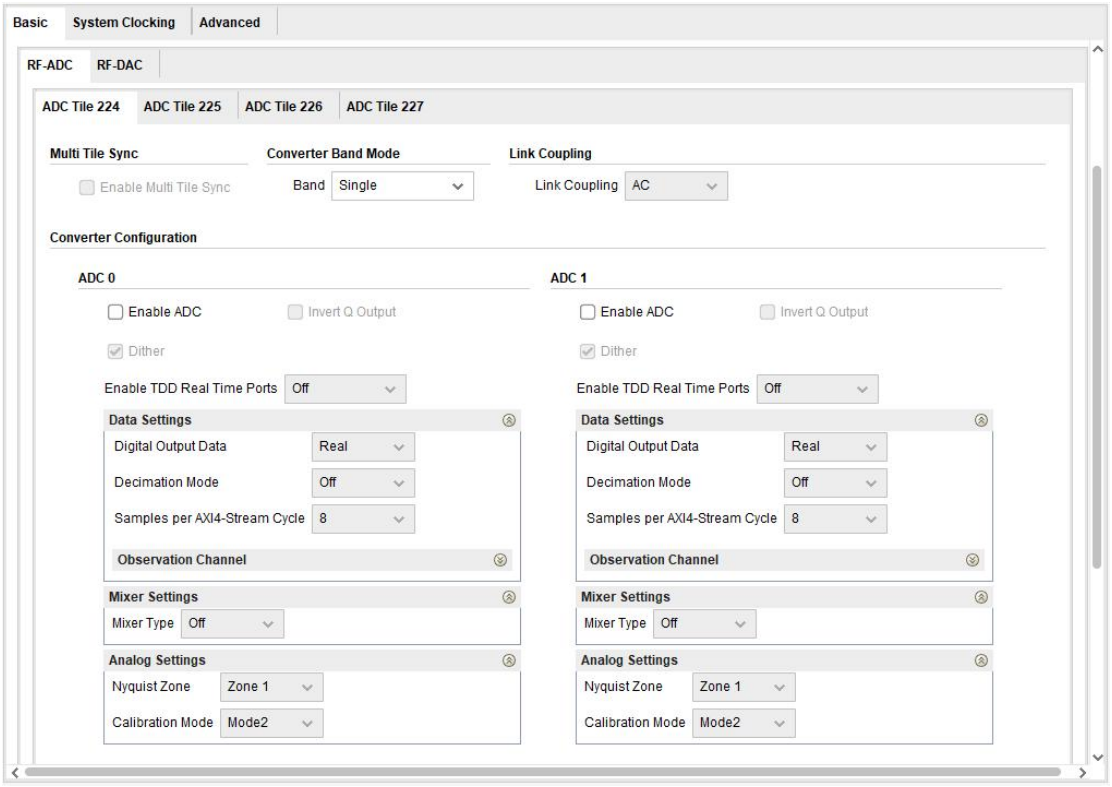
转换器设置菜单为设置 IP 配置提供了高级（Advanced）和简单（Simple）的选项。

Advanced：所有片都是独立配置的。

Simple：用户可以看到单个 RF-ADC 和 RF-DAC 块的配置选项。设置好片配置信息后，通过从给定的片列表中选择片，就可以将 RF-ADC 配置应用到设备中的任何 RF-ADC。类似地，RF-DAC 配置可以应用于任何 RF-DAC 片。

可以在不丢失配置信息的情况下从简单设置更改为高级设置。但是，从高级到简单是不可能的。只支持向下兼容。

RF-ADC 片设置



这些参数对片中每个 RF-ADC 都是通用的。

Link Coupling（链路耦合）：这决定了 RF-ADC 的输入信号是交流耦合还是直流耦合。典型的应用程序使用交流耦合，单对于某些应用程序（例如 Zero IF），也支持直流耦合。当选择直流耦合模式时，外部驱动电路的共模电压需要使用外部驱动的 VCM 引脚与 RF-ADC 内部的共模电压对齐。VCM 引脚只能用于直流耦合模式，如果选择交流耦合模式，则可以保持浮动。

Converter Band Mode（转换器频带模式）：每个 RF-ADC 可以独立工作（单频

带模式），也可以与其它转换器一起工作（多频带模式），以下模式可用于 RF-ADC。

Enable ADC（启用 ADC）：选择是否启用所选片内的所选转换器。

Invert Q Output（Q 逆变输出）：只有选择 I/Q 输出数据且开启微调混频器时才能设置。当设置时，混合器的求积输出为负。这允许生成-Q 数据。

Dither（抖动）：选择所选片是否启用了抖动。抖动应启用，除非采样率低于 0.75 倍的 RF-ADC 的最大采样率。

Enable TDD Real Time Ports（Gen 3）启用了 TDD 实时端口：启用后，tdd_mode 端口将添加到 IP。这可以通过关闭 RF-ADC 的部分来实现节能。

Digital Output Data（数字输出数据）：设置片内所选转换器的数据类型。该参数仅在启用转换器时可配置。有效值为 Real/IQ。当转换器 0 被设置为 I/Q 时，转换器 1 也必须被启用，当转换器 2 被设置为 I/Q 时，转换器 3 也必须被启用，否则配置无效。

Decimation Mode（抽取模式）：设置所选片内所选转换器的抽取值。该参数仅在转换器启用时可配置。从下拉菜单中选择值 1x、2x、3x、4x、5x、6x、8x、10x、12x、16x、20x、24x、40x。当转换器未启用时，该值为关闭。

Samples per AXI4-Stream Word（每 AXI4-Stream 的采样字数）：设置每个周期的字数。当特定转换器已启用时，此参数是可配置的。有效值介于 1 和 12 之间，可以使用下拉列表进行选择。取值范围取决于所选的采样率，以将 AXI4-Stream 时钟保持在规范范围内。所需的 AXI4-Stream 时钟是 IP 核的输入，并显示基于所选总线宽度的值。

Observation Channel（观测通道）：启用并设置观测通道。

Mixer Type（混频器类型）：设置要使用的混频器类型（coarse、fine）。可选择的选项取决于转换器数字输出数据字段的选择。

Mixer Mode（混频器模式）：设置所选转换器的混合器模式。该参数仅在启用转换器时可配置。混频器模式的选择取决于混频器类型和所选数字输出数据的格式。当实数据输出时，混频器被绕过。当 I/Q 数据输出混频器可以设置为 read 的 I/Q 或 I/Q 到 I/Q。

Coarse Mixer Frequency（粗混频器频率）：设置粗混频器的频率。混频器类型

选择为粗调时，此参数可配置。有效选项为 Fs/2、Fs/4、和-Fs/4。

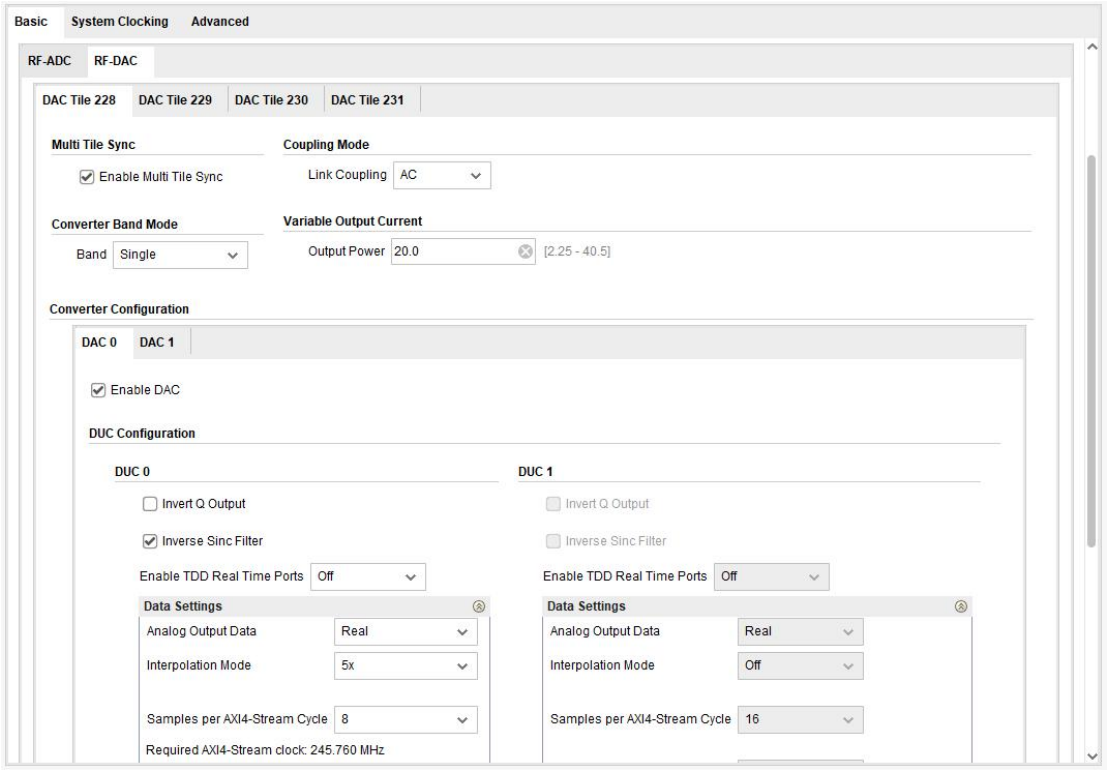
Fine Mixer Frequency（细混频器频率）： 设置细混频器的频率。此参数仅在选择 fine 作为混合器类型时有效。有效频率范围为-10GHz~10GHz。

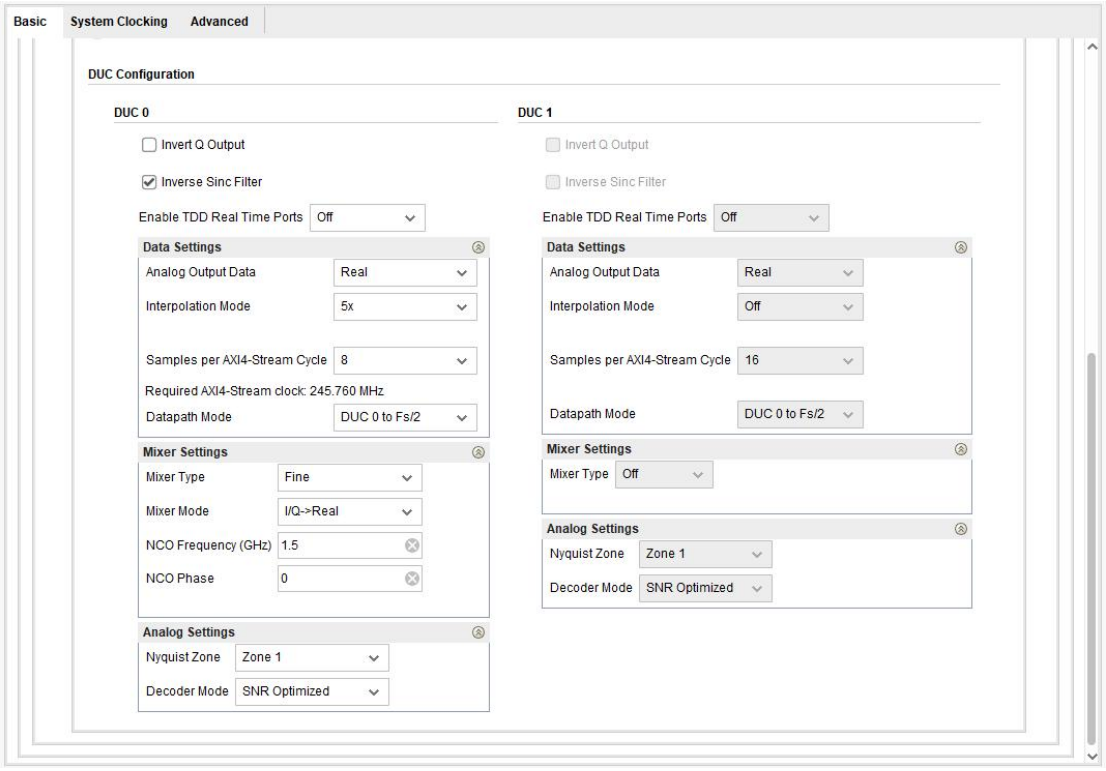
Fine Mixer Phase（细混频器相位）：设置细混频器的相位。此参数仅在选择 fine 作为混合器类型时有效。有效范围为-180 到 180。

Nyquist Zone（奈奎斯特区）： 选择偶数和奇数奈奎斯特区操作。

Calibration Mode（校准模式）：根据输入信号的特性选择不同的校准优化方案。自动模式（第三代）适用于所有输入频率。模式 1 是最佳输入频率 Fsamp/2（Nyquist）±10%。模式 2 是输入频率输出的最佳范围。

RF-DAC 片配置





这些参数对一个 tile 中的每个 RF-DAC 都是通用的。

Multi Tile Sync（多片同步）：启用后，块包含在多片同步组中。RF-DAC Tile0 必须启用并与被配置为启用此选项的 tile 的转换器 0 一起出现在组中。

Variable Output Current（Gen3）（可变输出电流（第三代））：当可变输出电流模式启用时，输出电流按照 DC 和 AC Switching 中的规定设置。DAC_AVTT 必须设置为 3.0V。

Link Coupling（链路耦合）：这决定了 RF-DAC 的输出信号是交流耦合还是直流耦合。在直流耦合模式下。共模电压是固定的。在此模式下，VOP 范围会减小。

RF-DAC 转换器配置

Common

Enable DAC（启用 DAC）：选择是否启用所选片内的所选转换器。取值为 TRUE 和 FALSE。

Invert Q Output（Q 逆变输出）：当使能时，转换器的 Q 输出被反转。仅当勾选“I/Q 输出数据”时可设置。后续的模拟混频器必须实现减法。1 x cos(f) - Q x

sin(f)，因此提供了一个-Q 信号（负正交）的选项，以便外部混频器可以执行加法，

I x cos(f) + (-Q)sin(f)。

Inverse Sinc Filter（Sinc 逆变滤波器）：选择是否启用 Sinc 逆变滤波器。该参数仅在启用转换器时可配置。取值为 TRUE 和 FALSE。这适用于两个 Nyquist 区域，基于模拟设置→Nyquist 区域选择。

Enable TDD Real Time Ports（Gen3）启用 TDD 实时端口：启用后，tdd_mode 端口将添加到 IP。这可以通过关闭 RF-DAC 的部分来实现功能。

Digital Output Data（数字输出数据）：设置所选片内所选转换器的数据类型。该参数仅在启用转换器时可配置。有效值为 Real 和 I/Q。当转换器 0 被设置为 I/Q 时，转换器 1 也必须被启用；当转换器 2 被设置为 I/Q 时，转换器 3 也必须被启用，否则配置无效。

Decimation Mode（抽取模式）：设置所选片内所选转换器的抽取值。该参数仅在转换器启用时可配置。从下拉菜单中选择值 1x、2x、3x、4x、5x、6x、8x、10x、12x、16x、20x、24x、40x。当转换器未启用时，该值为关闭。

Samples per AXI4-Stream Word（每 AXI4-Stream 的采样字数）：设置每个周期的字数。当特定转换器已启用时，此参数是可配置的。有效值介于 1 和 12 之间，可以使用下拉列表进行选择。取值范围取决于所选的采样率，以将 AXI4-Stream 时钟保持在规范范围内。所需的 AXI4-Stream 时钟是 IP 核的输入，并显示基于所选总线宽度的值。

Datapath Mode（Gen3）数据路径模式：选择 RF-DAC 数据路径模式。以下选项是可用的。这些模式适用于数字数据路径，可以使用模拟设置→奈奎斯特区域选择来自第一或第二奈奎斯特区域输出该数据。

DUC 0 to Fs/2：全奈奎斯特特 DUC 模式。

DUC 0 to Fs/4：使用低通 IMR 滤波器的半奈奎斯特 DUC 模式。

DUC Fs/4 to Fs/2：使用高通 IMR 滤波器的半奈奎斯特 DUC 模式。

No DUC 0 to Fs/2：DU 旁路模式。

Mixer Settings

Mixer Type（混频器类型）：设置要使用的混频器类型。有效的选项有 bypassed、coarse and fine。可选择的选项取决与转换器数字输出数据字段的选择。

Mixer Mode（混频器模式）：设置所选转换器的混合器模式。该参数仅在启用转换器时可配置。混频器模式的选择取决于混频器类型和所选数字输出数据的格式。当真实数据输出时，混频器被绕过。当 I/Q 数据输出混频器可以设置为 real 的 I/Q 或 I/Q 到 I/Q。

Coarse Mixer Frequency（粗混频器频率）：设置粗混频器的频率。混频器类型选择为粗调时，此参数可配置。有效选项为 Fs/2\、Fs/4 和-Fs/4。

Fine Mixer Frequency（细混频器频率）：设置细混频器的频率。此参数仅在选择 fine 作为混频器类型时有效。有效范围额外：-10GHz~10GHz。

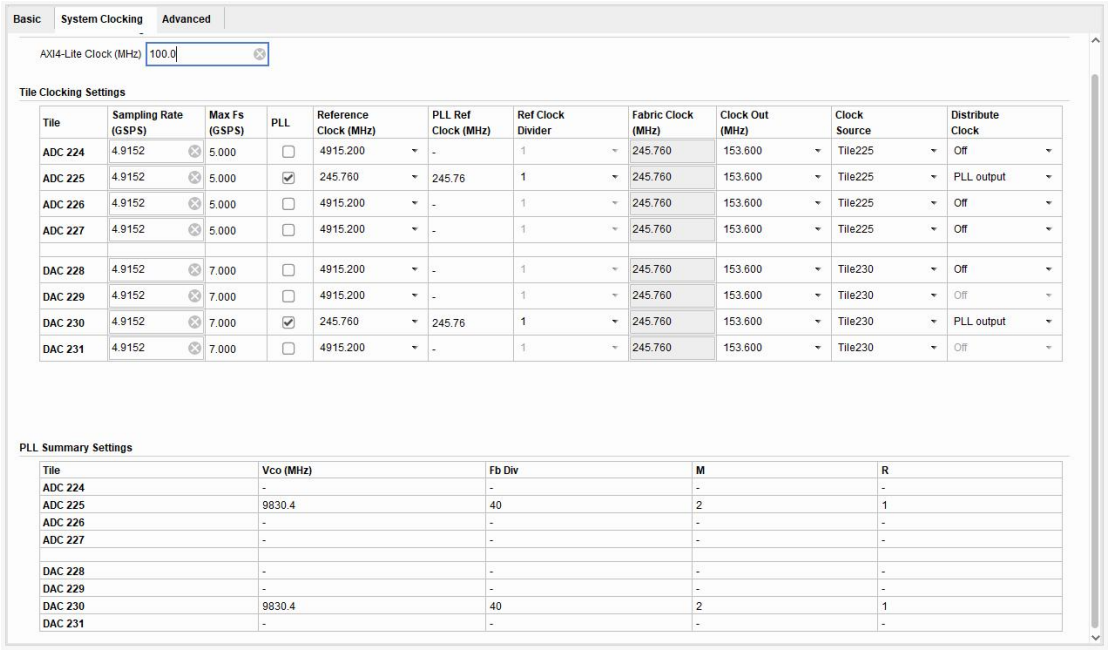
Fine Mixer Phase（细混频器相位）：设置细混频器的相位。此参数仅在选择 fine 作为混合器类型时有效。有效范围时-180 到 180。

Analog Settings

Nyquist Zone（奈奎斯特区）：选择偶数和奇数奈奎斯特区操作。

Decode Mode（解码模式）：RF-DAC 解码器可以优化为低信噪比或高线性度。

系统时钟配置



AXI4-Lite 接口配置

AXI Clock Frequency（MHz）：核心需要关于 AXI4-Lite 时钟输入频率的信息，以确保 RF-ADC 和 RF-DAC 块的上电顺序的正确计时。时钟的速度应该以兆赫为单位输入。

片时钟设置

采样率和时钟频率在本节中设置。这些设置由片中的每个转换器共享。

Sampling Rate（GSPS）：设置每个片的采样率。当片中至少有一个转换器被启用时，它是可配置的。有效值取决于所选的设备和包。

Max Fs（GSPS）：给出了每个片的最大采样率以供参考。

PLL：选择是使用还是屏蔽片中的 PLL。当片中的至少一个转换器被启用时，它时可以配置的。如果需要使用 RF dc 驱动程序 API 进行动态重新配置 PLL，则必须在 Vivado IDE 中启用 PLL。有效值为 True 或 False。

Reference Clock（MHz）：设置片的时钟输入频率。它是可配置时，锁相环和至少一个转换器在 tile 是启用的。他的值取决于 tile 的采样率。给出了一个基于所选抽样率的值的下拉列表。为了获得最佳的相位噪声性能，建议在锁相环输入处理使用正确的参考频率。

PLL Reference Clock：设置频率的 PLL 输入时钟，如果 PLL 是为了一个特定的 tile 启用。

Reference Clock Divider：参考时钟分频器，设置锁相环参考时钟输入上的分频器。

Fabric Clock（MHz）：这是在所选片的 AXI4-Stream 时钟输入上提供的时钟频率。所需的频率由采样率和转换器配置中的位置决定。因为一个片上的所有 AXI4-Stream 端口共享一个公共的 AXI4-Stream 时钟，所以片上的所有转换器配置必须要求相同的时钟频率。

Clock Out（MHz）：设置片上输出时钟的频率。这个时钟可以用来驱动 AXI4-Stream 时钟输入。当 tile 中至少有一个转换器被启用时，它是可配置的。这些值取决于 tile 的采样率。给出了一个基于所选抽样率的值的下拉列表。

Clock Source（第三代）：设置每个片的时钟源。对于只有双 RF-DAC 的第三代设备，每个启用 RF-DAC 的时钟源必须设置为偶数块。

Distribute Clock（第三代）：设置片是否将分配时钟到设备上的其它片。

Off：不进行分配时钟。

Input Refclk：当 PLL 启用时，输入参考时钟从 tile 转发。如果 PLL 未启用，则分配输入采样时钟。

PLL Output：当 PLL 被启用时，来自 PLL 的时钟输出从 tile 转发。

锁相环概要设置

PLL 摘要提供有关已启用 PLL 的频率和分频器设置的信息。

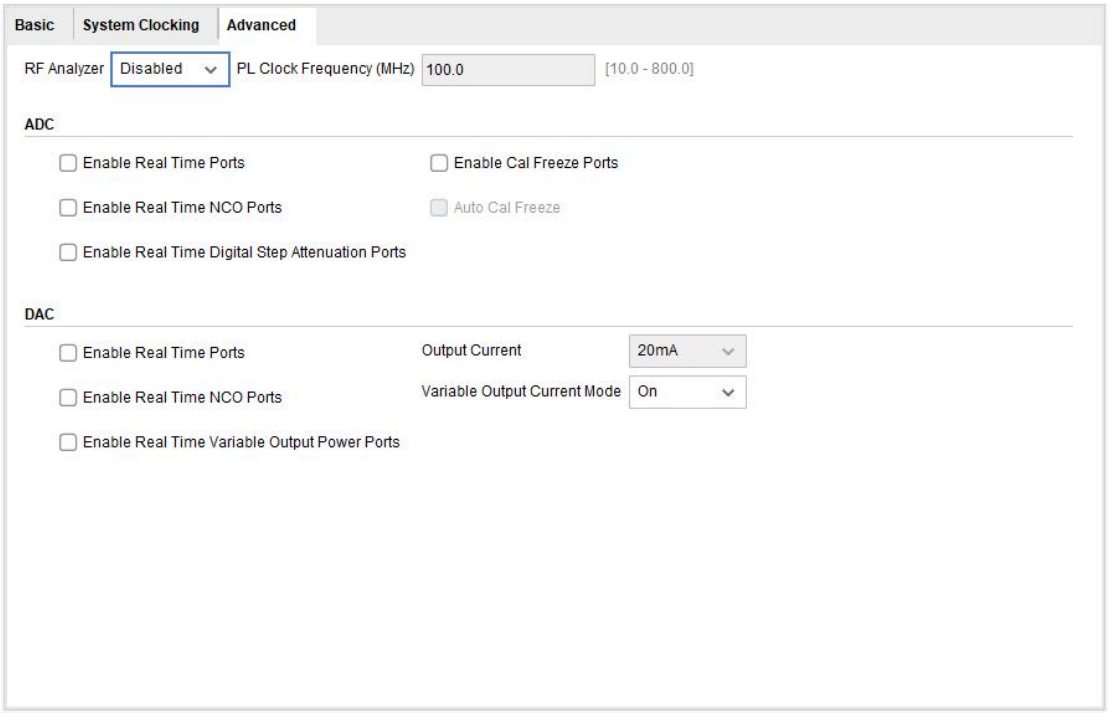
Vco（MHz）：压控振荡器的频率。

Fb Div：反馈分频器的设置。

M：输出分频器设置。

R：参考时钟分频器设置。

高级配置



RF Aanlyzer：射频分析仪提供硬件测试系统。系统包含为这个 IP 核实例配置的数据刺激和捕获快。

PL Clock Frequency（MHz）：如果选择多片同步，则必须提供 PLL 时钟和向

系统发送 PL SYSREF。为了准确的 SYSREF 捕获，频率必须是 RF-ADC 和 RF-DAC AXI4-Stream 时钟的普通整数倍。

RF-ADC

Enable Real Time Ports：使能 RF-ADC Over Threshold、Over Voltage、Over Range 和 pl_event 端口。

Enable cal Freeze Ports：使能校准冻结逻辑，当模拟输入过压或相关的 int_cal_freeze 输入被断言时，校准被冻结。

Auto Cal Freeze：自动校准冻结。如果选择此选项，除了手动校准冻结过程之外，当模拟输入超出范围时，校准将被冻结。

Enalbe Real Time NCO Ports：启用实时 NCO 端口。可以实时修改 DDC 中混频器 NCO 的频率和相位。选择此选项可启用 RF-ADC 的实时 NCO 接口。

Enable Real Time Digital Step Attenuation Ports（Gen3）：启用实时数字步进衰减端口（第三代）。如果选择此选项，则可以通过实时 DSA 信号接口更改数字步进衰减器设置。

RF-DAC

Enable Real Time Ports：启用 RF-DAC Fast Shutdown 和 pl_event 端口。

Enable Real Time NCO Ports：在 DUC 中，混频器的频率和相位可以实时修改。选择此选项将启用 RF-DAC 的实时 NCO 端口。

Output Current：选择 20mA 和 32mA 的 RF-DAC 输出电流。在 20mA 模式下，DAC_AVTT 等于 2.5V。在 32mA 模式下，DAC_AVTT 等于 3.0V。

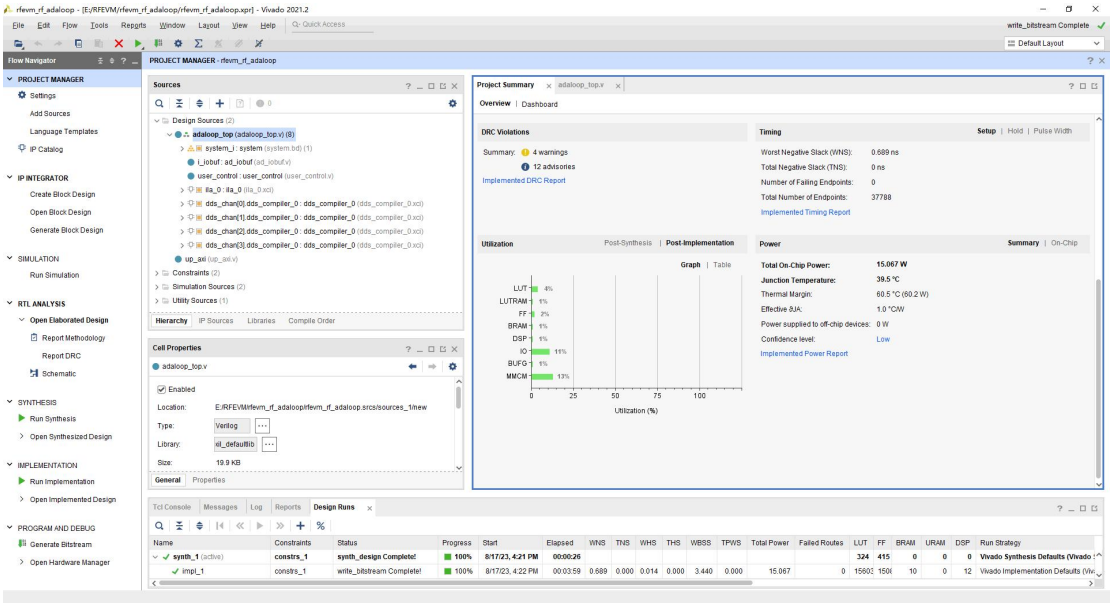
Variable Output Current Mode（Gen3）可变输出电流模式（第三代）：当可变输出电流模式关闭时，输出电流由输出电流选项设置。在 20mA 模式下，DAC_AVTT 等于 2.5V。在 32mA 模式下，DAC_VATT 等于 3.0V。启用可变输出模式后，可以通过 RF-DAC 选项卡中的可变输出电流字段在每个片的基础上设置输出电流。DAC_AVTT 设置为 3.0V。

Enable Real Time Variable Output Power Ports（Gen3）：启用实时可变输出电源端口（第三代），如果选择此选项，则会为每个启用的 RF-DAC 添加实时 VOP 信号接口。这使得能够实时更新输出功率。

2.3 FPGA 硬件工程介绍

FPGA 硬件工程通过 Vivado2021.2 搭建，包括 PL 逻辑部分和 PS 处理器配置部分。通过 SPI 配置 LMK04828 时钟芯片、Axi-Lite 配置 AD/DA 核、Axi-Lite 读写 PS PL 交互寄存器。

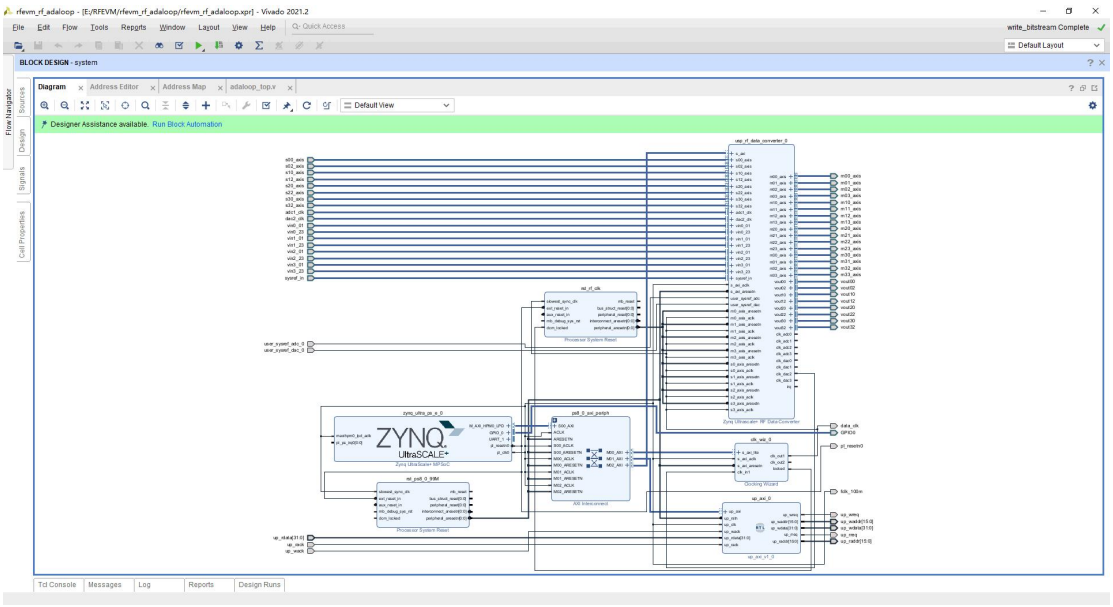
Vivado 打开工程，界面如下所示：



文件解析：

文件名	层级	功能
adaloop_top	0	顶层文件，例化子模块
system	1	BD 文件，包含 PS 和 RF 转换器模块
ad_iobuf	1	差分转单端模块
user_control	1	寄存器读写模块
dds_complier_0	1	dds ip 核，产生 DA 源
ila_0	1	Chipscope,观测 AD 信号

我们在 Vivado 软件里面打开 BD 文件 system，界面如下所示



里面主要三个模块 Zynq UltraScale+ MPSoC、usp_rf_data_converter_0 和 up_axi 模块。

模块名称	功能
Zynq UltraScale+ MPSoC	PS 部分，里面包含 PS 的外设配置
usp_rf_data_converter_0	RF 转换器模块，包含 AD/DA 的初始化配置
up_axi	PS 通过 axi_lite 总线读写 PL 寄存器模块

其中，Zynq UltraScale+ MPSoC IP 核配置可以参考“RFEVM 开发板 FPGA 基础教程”里面 IPI 设计部分，去配置相关参数。

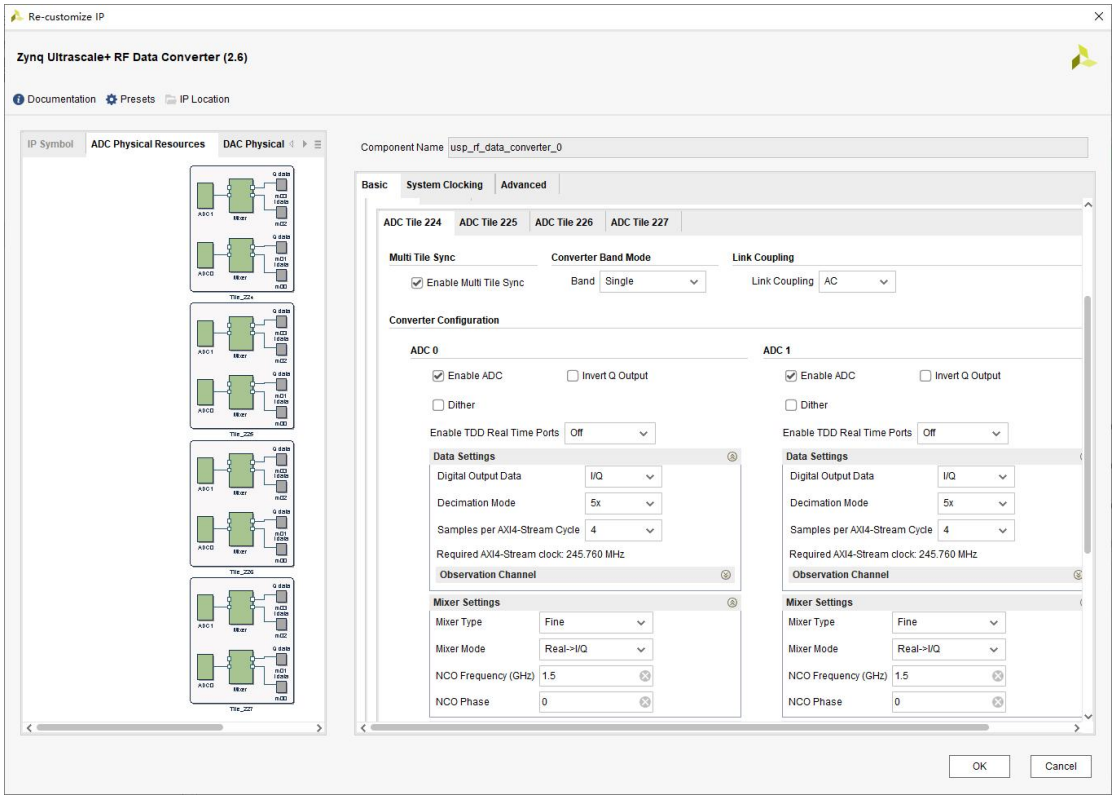
RF 转换器配置

usp_rf_data_converter_0 是 RFSoC 的 AD/DA 硬核配置 IP 模块，用以配置时钟链路、AD/DA 模式和 MTS 同步模式，这里面大多数参数可以通过 vitis/SDK 二次配置。

配置界面如下：

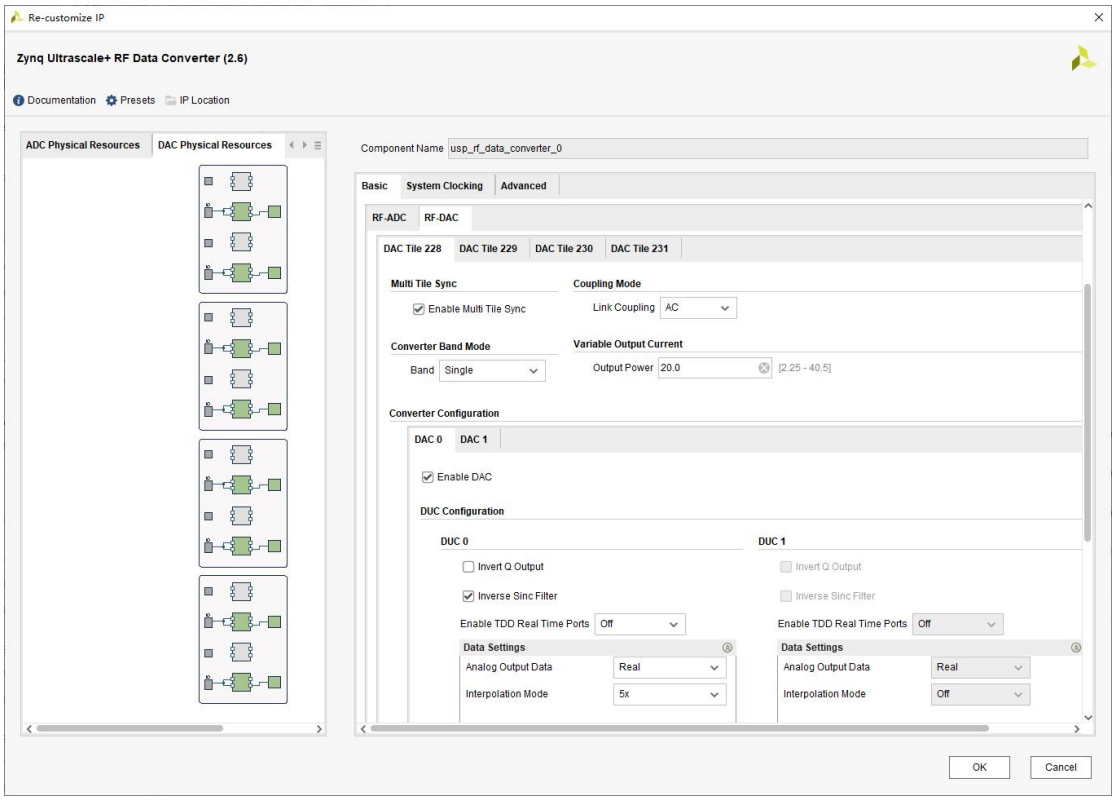
RF-ADC 配置

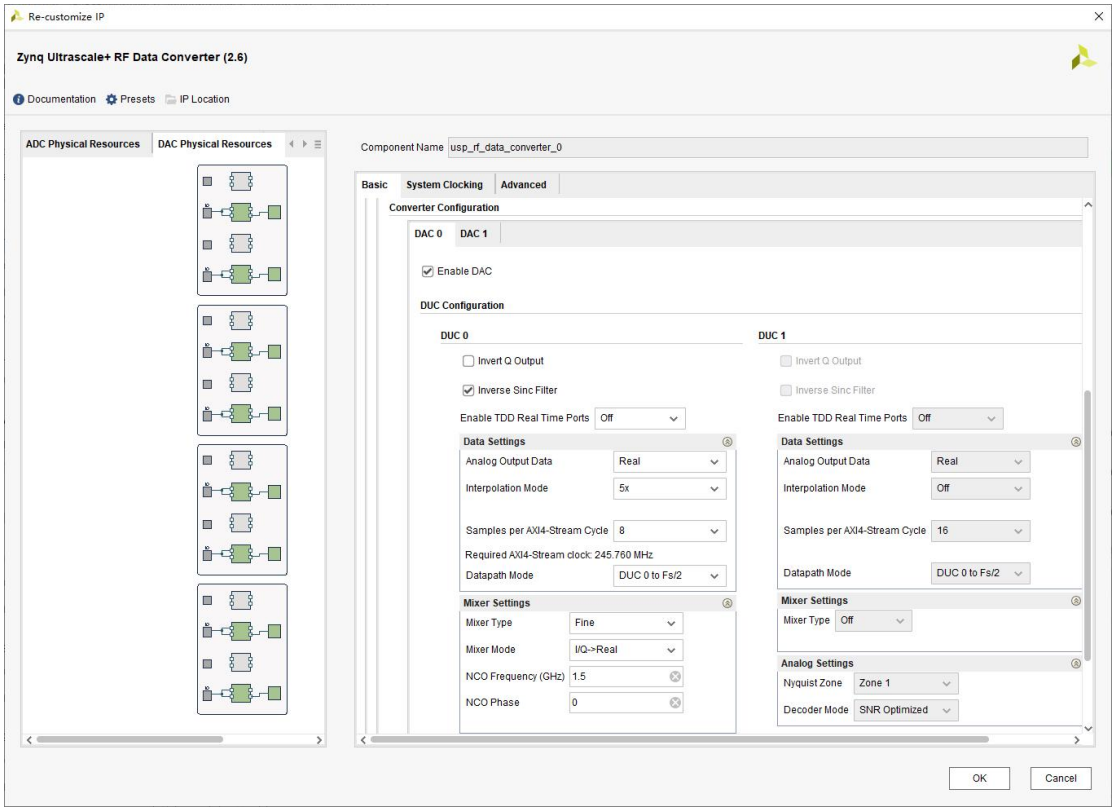
默认设定为 mixer 模式，射频实数输入，基带 IQ 输出。其他参数如频点范围、抽取系数等都可以二次配置。



RF-DAC 配置

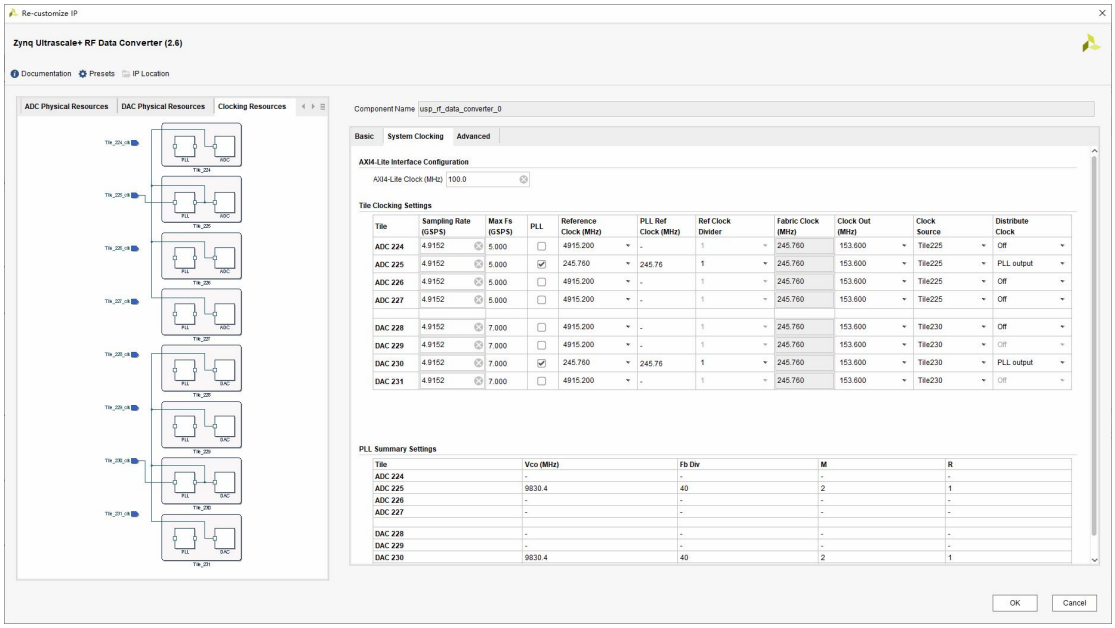
默认设定为 mixer 模式，基带 IQ 输入，射频实数输出。其他参数如频点范围，抽取系数等都可以二次配置。





System Clcoking 配置

RFSoC 第三代支持多个 tile 共享参考时钟，这样对 MTS 多通道同步的实现更加友好，本设计采用这种模式。四个发送通道共享 Bank250 参考时钟，四个接受通道共享 Bank225 参考时钟。ADC/DAC 的采样率是 4.9251GHz，通过 LMK04828 时钟芯片输出的 245.76MHz 参考时钟锁相倍频实现。



Clock Out（MHz）选择 153.6MHz，axi_stream 总线的时钟计算与匹配的方式如

下：

ADC/DAC 的采样率为 4.9152GHz；

DAC 五倍插值，总线上一个时钟 8 个样点，则总线时钟为 4.9152/5/4=245.76MHz

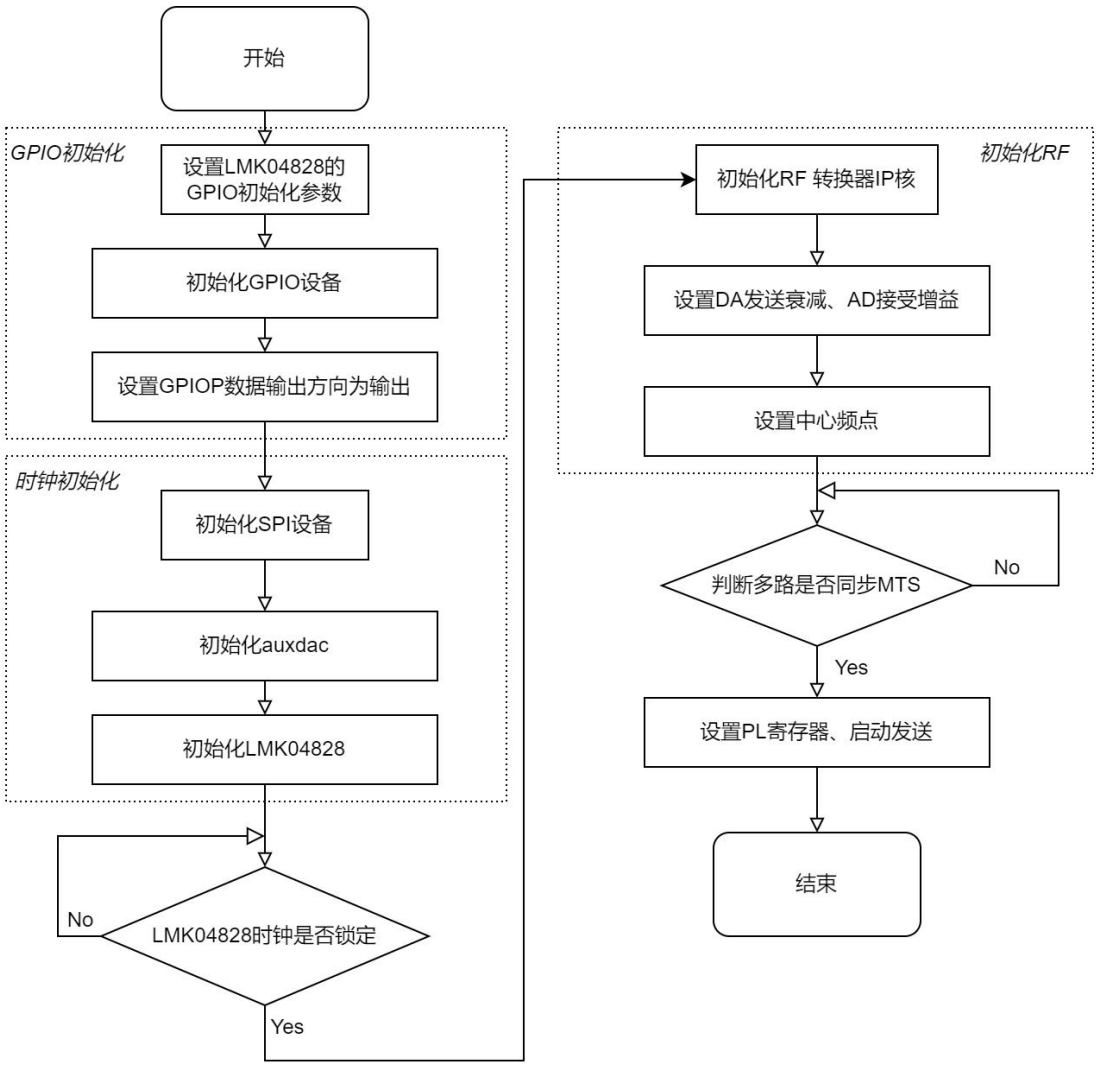
ADC 五倍抽取，总线上一个时钟 4 个样点（注意这里 ADC 是 IQ 总线独立，所以一个时钟还是 8 个样点）总线时钟为 4.9152/5/4=245.76MHz。

设计中采用一个 MMCM 把 ADC/DAC IP 输出的 153.6MHz 倍频到 245.76MHz 给基带系统做同步时钟。当配置不同的插值和抽取值，总线的数据位宽是恒定的（8 个），总线的时钟会有变化，则我们只需要配置 MMCM 的系数即可以动态实现多种采样率的变化。

2.4 裸机应用程序

裸机工程主要用来初始化 LMK04828 时钟芯片、初始化 RFSoC 配置、GPIO 配置、与 PL 寄存器交互。

软件配置流程如下图所示：



1) 首先执行 platform_init(void)进行 GPIO 设备初始化设置方向并设定默认值。

```
    print("*************************************************\n\r");
    print("********RFEVM CH8 trxloop begin MOD RFSOM*********\n\r");
    print("*************************************************\n\r");
    printf("Please wait...\n");
    platform_init();
    gpio_set_value(TRX_SW, 1);
    gpio_set_value(FDDTDD_SW, 1);
    gpio_set_value(LMK_RESET, 0);
    mdelay(500);
    gpio_set_value(LMK_REF_SELECT0, 1); //CLKIN1 EXTERNAL
    gpio_set_value(LMK_REF_SELECT1, 0);
    gpio_set_value(LMK_SYNC, 0);
```

2) 配置LMK04828输出RFSoC所需设备时钟245.76MHz与SYSREF（0.96MHz）时钟。

```
    /*****************************************************************
****/
    /*****      configure system clock
*****/
    /*****************************************************************
****/
    int32_t val;
    val = iic_init(IIC_DEVICE_ID);
    if (val == 0) {
        printf("IIC Setup PASSED\r\n");
        uint8_t iicdata[2];
        for (int i = 0; i < sizeof(iicdata); i++) {
            val = i2c_reg_read(0x50, i, &iicdata[i]);
            mdelay(10);
        }
        auxdac = iicdata[0] | (iicdata[1] << 8);
        printf("auxdac value read from eeprom %d\n", auxdac);
        if (auxdac >= 2000 || auxdac < 1000)
            auxdac = 1600;
    }
    lmk04828_init(&lmk04828_spi, 245.76e6);
    lmk04828_update_man_dac(&lmk04828_spi, auxdac);
    int32_t lock_pll1 = -1;
    int32_t lock_pll2 = -1;
    int cyc = 0;
    while ((lock_pll1 != ref_ext_en) || lock_pll2 != 1 || cyc < 100) {
        lock_pll1 = lmk04828_spi_read(&lmk04828_spi, 0X182) >> 1 & 0x1;
        lock_pll2 = lmk04828_spi_read(&lmk04828_spi, 0X183) >> 1 & 0x1;
        //printf("read from rf1_lmk04828 pll1_lock=0x%lx\n", lock_pll1);
        printf("read from rf1_lmk04828 pll2_lock=0x%x\n", lock_pll2);
        if (lock_pll2 == 1 && (lock_pll1 == ref_ext_en)) {
            lock_pll2 = 1;
            lock_pll1 = ref_ext_en;
            cyc++;
        } else {
            lock_pll2 = 0;
            lock_pll1 = (~ref_ext_en) & 0x1;
            cyc = 0;
        }
    }
    printf("read from rf1_lmk04828 pll1_lock=0x%x\n", lock_pll1);
    printf("read from rf1_lmk04828 pll2_lock=0x%x\n", lock_pll2);
    gpio_set_value(LMK_SYNC, 0);
    mdelay(1);
```

```
    gpio_set_value(LMK_SYNC, 1);
    mdelay(1);
    gpio_set_value(LMK_SYNC, 0);
    lmk04828_update_sysref_pulse(&lmk04828_spi);
    gpio_set_value(LMK_SYNC, 0);
    lmk04828_set_sysref_shots(&lmk04828_spi, 0);
    mdelay(1);
    mdelay(500);
```

3) RFSoC 初始化

```
Status = RFdcDcp_initial(&RFdcInstPtr, RFDC_DEVICE_ID);
```

设定接受的 DSA 衰减器，SettingsPtr.Attenuation=0 是不衰减，最大衰减为 27dB

设定发送的 DAC 电流，40500（uA）是最大输出发射功率，最小为 2250uA，对应的动态范围 25dB

```
    SettingsPtr.Attenuation = 0;
    SettingsPtr.DisableRTS = 1;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 2; j++) {
            Status = XRFdc_SetDACVOP(&RFdcInstPtr, i, j * 2, txatt[i * 2 + j]);
            printf("tile=%d,block=%d,XRFdc_SetDACVOP,Status=%d\n", i, j,
Status);
            Status = XRFdc_SetDSA(&RFdcInstPtr, i, j, &SettingsPtr);
            Status = XRFdc_GetDSA(&RFdcInstPtr, i, j, &SettingsPtr);
            Status = XRFdc_SetDither(&RFdcInstPtr, i, j, 1);
            printf("tile=%d,block=%d,Attenuation=%f,Status=%d\n", i, j,
                    SettingsPtr.Attenuation, Status);
        }
```

检测参考时钟

```
    mdelay(1);
    uint32_t val_out;
    val_out = AXI_REG_READ(XPAR_XRFDC_0_BASEADDR, 0);
    printf("ip revision=%x\n", val_out);
    val_out = AXI_REG_READ(XPAR_XRFDC_0_BASEADDR, 0x18084);
    printf("adc tile1 clock_dector=%x\n", val_out);
    val_out = AXI_REG_READ(XPAR_XRFDC_0_BASEADDR, 0x1c084);
    printf("adc tile2 clock_dector=%x\n", val_out);
    val_out = AXI_REG_READ(XPAR_XRFDC_0_BASEADDR, 0x04084);
    printf("dac tile0 clock_dector=%x\n", val_out);
    val_out = AXI_REG_READ(XPAR_XRFDC_0_BASEADDR, 0x0c084);
```

```
    printf("dac tile2 clock_dector=%x\n", val_out);
    FS = RFdcInstPtr.ADC_Tile[0].PLL_Settings.SampleRate * 1e9;
    printf("DAC converter samplerate=%ldHz\n", FS);
    printf("ADC converter samplerate=%ldHz\n", FS);
```

设定 DAC 发送频点

```
    double Freq_set = 1500;
    uint32_t nco_nyquist;
    if (Freq_set * 1e6 < FS / 2)
        nco_nyquist = 1;
    else {
        nco_nyquist = 2;
        Freq_set = -Freq_set;
    }
    for (Tile_Id = 0; Tile_Id < 4; Tile_Id++) {
        for (Block_Id = 0; Block_Id < 2; Block_Id++) {
            Status = XRFdc_SetNyquistZone(&RFdcInstPtr, XRFDC_DAC_TILE,
Tile_Id,
                    Block_Id * 2, nco_nyquist);
            if (Status != XST_SUCCESS)
                printf("Tile_Id=%d,Block_Id=%d,XRFdc_SetNyquistZone fail\n",
                        Tile_Id, Block_Id);
            Status = XRFdc_ChangeNCOFreq(&RFdcInstPtr, XRFDC_DAC_TILE,
Tile_Id,
                    Block_Id * 2, Freq_set);
            if (Status != XST_SUCCESS)
                printf("Tile_Id=%d,Block_Id=%d,XRFdc_ChangeNCOFreq fail\n",
                        Tile_Id, Block_Id);
            Status = XRFdc_GetMixerSettings(&RFdcInstPtr, XRFDC_DAC_TILE,
Tile_Id,
                    Block_Id * 2, &Mixer_Settings);
            if (Status != XST_SUCCESS)
                printf("Tile_Id=%d,Block_Id=%d,XRFdc_GetMixerSettings
fail\n",
                        Tile_Id, Block_Id);
            printf("DAC NCO frequency of Tile#%d Block%d is set to %f\n",
                    Tile_Id, Block_Id, Mixer_Settings.Freq);
        }
    }
```

设定 ADC 接受频点

```
    Freq_set = 1500;
```

```
    if (Freq_set * 1e6 < (FS / 2)) {
        nco_nyquist = 1;
        Freq_set = -Freq_set;
    } else {
        nco_nyquist = 2;
        Freq_set = Freq_set;
    }
    for (Tile_Id = 0; Tile_Id < 4; Tile_Id++) {
        for (Block_Id = 0; Block_Id < 2; Block_Id++) {
            Status = XRFdc_SetNyquistZone(&RFdcInstPtr, XRFDC_ADC_TILE,
Tile_Id,
                    Block_Id, nco_nyquist);
            if (Status != XST_SUCCESS)
                printf("Tile_Id=%d,Block_Id=%d,XRFdc_SetNyquistZone fail\n",
                        Tile_Id, Block_Id);
            Status = XRFdc_ChangeNCOFreq(&RFdcInstPtr, XRFDC_ADC_TILE,
Tile_Id,
                    Block_Id, Freq_set);
            if (Status != XST_SUCCESS)
                printf("Tile_Id=%d,Block_Id=%d,XRFdc_ChangeNCOFreq fail\n",
                        Tile_Id, Block_Id);
            Status = XRFdc_GetMixerSettings(&RFdcInstPtr, XRFDC_ADC_TILE,
                    Tile_Id, Block_Id, &Mixer_Settings);
            if (Status != XST_SUCCESS)
                printf("Tile_Id=%d,Block_Id=%d,XRFdc_GetMixerSettings
fail\n",
                        Tile_Id, Block_Id);
            printf("ADC NCO frequency of Tile#%d Block%d is set to %f\n",
                    Tile_Id, Block_Id, Mixer_Settings.Freq);
        }
    }
```

设定插值和抽取系数，同步修改锁相环生成总线时钟

```
    sample_rate = 245.76e6;
    AXI_REG_WRITE(axi_lite_addr, 4 * 4, sample_rate / 4); //clock_1pps
    Status = ChangeSampleFreq(&RFdcInstPtr, sample_rate);
```

设定接受 ADC 的 QMC 增益，QMC 增益必须小于 2，这里设置 1.99≈6dB 增益。

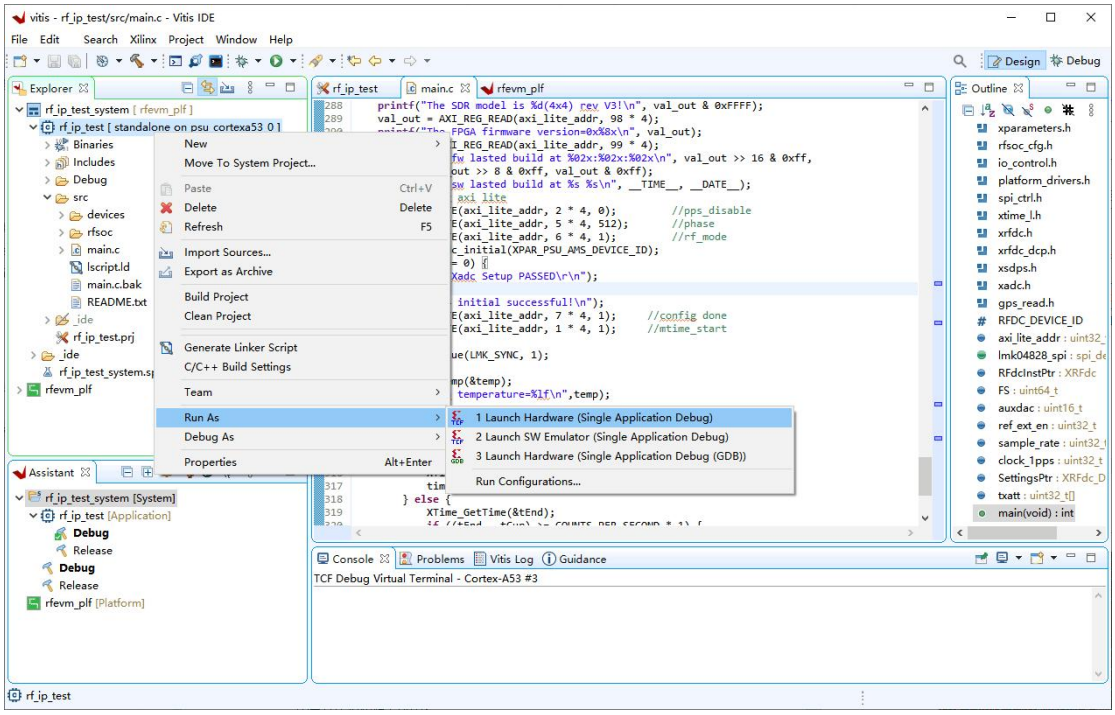
```
Status = qmc_settings(&RFdcInstPtr, XRFDC_ADC_TILE);
```

4) 配置 PL 里面的寄存器，使能 DDS 产生 DA 测试源

2.5 运行 demo 例程

1) 连接 12v 直流电源，Type-C 调试线缆插好，板卡上电；

2) Vitis 中直接运行应用程序；



注：vitis 工程对路径比较敏感，当 vitis 工程复制到其他路径后，编译或者下载会提示一些错误，这时简单的做法可以在 vitis 里面把 rfevm_plf 删掉，重新建个 platform，起名 rfevm_plf，里面参数默认即可。xsa 文件选工程下面的文件按即可。

3) 打开串口调试终端，串口配置为：波特率 115200，8 数据位，1 开始，1 停止；

串口打印信息如下所示：

```
********
********RFEVM CH8 trxloop begin MOD RFSOM**********
**************************************************
Please wait...
IIC Setup PASSED
auxdac value read from eeprom 65535
rf:1,total register number=128
rf:1,lmk04828_reg[6]=20
man_dac=1600,dac_value=512
read from rf1_lmk04828 pll2_lock=0x1
read from rf1_lmk04828 pll2_lock=0x1
read from rf1_lmk04828 pll2_lock=0x1
```

```
read from rf1_lmk04828 pll2_lock=0x1
read from rf1_lmk04828 pll2_lock=0x1
read from rf1_lmk04828 pll2_lock=0x1
read from rf1_lmk04828 pll1_lock=0x0
read from rf1_lmk04828 pll2_lock=0x1
CRL_APB_BOOT_MODE_USER=e
ADC_PATHS_ENABLED=ffffffff
DAC_PATHS_ENABLED=55555555
tile=0,block=0,XRFdc_SetDACVOP,Status=0
tile=0,block=0,Attenuation=0.000000,Status=0
tile=0,block=1,XRFdc_SetDACVOP,Status=0
tile=0,block=1,Attenuation=0.000000,Status=0
tile=1,block=0,XRFdc_SetDACVOP,Status=0
tile=1,block=0,Attenuation=0.000000,Status=0
tile=1,block=1,XRFdc_SetDACVOP,Status=0
tile=1,block=1,Attenuation=0.000000,Status=0
tile=2,block=0,XRFdc_SetDACVOP,Status=0
tile=2,block=0,Attenuation=0.000000,Status=0
tile=2,block=1,XRFdc_SetDACVOP,Status=0
tile=2,block=1,Attenuation=0.000000,Status=0
tile=3,block=0,XRFdc_SetDACVOP,Status=0
tile=3,block=0,Attenuation=0.000000,Status=0
tile=3,block=1,XRFdc_SetDACVOP,Status=0
tile=3,block=1,Attenuation=0.000000,Status=0
ip revision=2060000
adc tile1 clock_dector=1
adc tile2 clock_dector=1
dac tile0 clock_dector=1
dac tile2 clock_dector=1
DAC converter samplerate=4915200000Hz
ADC converter samplerate=4915200000Hz
DAC NCO frequency of Tile#0 Block0 is set to 1500.000000
DAC NCO frequency of Tile#0 Block1 is set to 1500.000000
DAC NCO frequency of Tile#1 Block0 is set to 1500.000000
DAC NCO frequency of Tile#1 Block1 is set to 1500.000000
DAC NCO frequency of Tile#2 Block0 is set to 1500.000000
DAC NCO frequency of Tile#2 Block1 is set to 1500.000000
DAC NCO frequency of Tile#3 Block0 is set to 1500.000000
DAC NCO frequency of Tile#3 Block1 is set to 1500.000000
ADC NCO frequency of Tile#0 Block0 is set to -1500.000000
ADC NCO frequency of Tile#0 Block1 is set to -1500.000000
ADC NCO frequency of Tile#1 Block0 is set to -1500.000000
ADC NCO frequency of Tile#1 Block1 is set to -1500.000000
ADC NCO frequency of Tile#2 Block0 is set to -1500.000000
ADC NCO frequency of Tile#2 Block1 is set to -1500.000000
ADC NCO frequency of Tile#3 Block0 is set to -1500.000000
ADC NCO frequency of Tile#3 Block1 is set to -1500.000000
XRFdc_SetInterpolationFactor Tile#0 Block0=20
XRFdc_SetInterpolationFactor Tile#0 Block2=20
XRFdc_SetInterpolationFactor Tile#1 Block0=20
XRFdc_SetInterpolationFactor Tile#1 Block2=20
XRFdc_SetInterpolationFactor Tile#2 Block0=20
```

```
XRFdc_SetInterpolationFactor Tile#3 Block2=20
XRFdc_SetDecimationFactor Tile#0 Block0=20
XRFdc_SetDecimationFactor Tile#0 Block1=20
XRFdc_SetDecimationFactor Tile#1 Block0=20
XRFdc_SetDecimationFactor Tile#1 Block1=20
XRFdc_SetDecimationFactor Tile#2 Block0=20
XRFdc_SetDecimationFactor Tile#2 Block1=20
XRFdc_SetDecimationFactor Tile#3 Block0=20
XRFdc_SetDecimationFactor Tile#3 Block1=20
XRFdc_GetInterpolationFactor Tile#0 Block0=20
XRFdc_GetInterpolationFactor Tile#0 Block1=20
XRFdc_GetInterpolationFactor Tile#1 Block0=20
XRFdc_GetInterpolationFactor Tile#1 Block1=20
XRFdc_GetInterpolationFactor Tile#2 Block0=20
XRFdc_GetInterpolationFactor Tile#2 Block1=20
XRFdc_GetInterpolationFactor Tile#3 Block0=20
XRFdc_GetInterpolationFactor Tile#3 Block1=20
XRFdc_GetDecimationFactor Tile#0 Block0=20
XRFdc_GetDecimationFactor Tile#0 Block1=20
XRFdc_GetDecimationFactor Tile#1 Block0=20
XRFdc_GetDecimationFactor Tile#1 Block1=20
XRFdc_GetDecimationFactor Tile#2 Block0=20
XRFdc_GetDecimationFactor Tile#2 Block1=20
XRFdc_GetDecimationFactor Tile#3 Block0=20
XRFdc_GetDecimationFactor Tile#3 Block1=20
clock lock=0
clock lock=1
mmcm clock rate is 61440000Hz
tile=0,block=0,EnableGain=0
tile=0,block=0,GainCorrectionFactor=0.000000
tile=0,block=1,EnableGain=0
tile=0,block=1,GainCorrectionFactor=0.000000
tile=1,block=0,EnableGain=0
tile=1,block=0,GainCorrectionFactor=0.000000
tile=1,block=1,EnableGain=0
tile=1,block=1,GainCorrectionFactor=0.000000
tile=2,block=0,EnableGain=0
tile=2,block=0,GainCorrectionFactor=0.000000
tile=2,block=1,EnableGain=0
tile=2,block=1,GainCorrectionFactor=0.000000
tile=3,block=0,EnableGain=0
tile=3,block=0,GainCorrectionFactor=0.000000
tile=3,block=1,EnableGain=0
tile=3,block=1,GainCorrectionFactor=0.000000
tile=0,block=0,EnableGain=1,Status=0
tile=0,block=0,GainCorrectionFactor=1.989990
tile=0,block=1,EnableGain=1,Status=0
tile=0,block=1,GainCorrectionFactor=1.989990
tile=1,block=0,EnableGain=1,Status=0
tile=1,block=0,GainCorrectionFactor=1.989990
tile=1,block=1,EnableGain=1,Status=0
tile=1,block=1,GainCorrectionFactor=1.989990
```

```
tile=3,block=1,GainCorrectionFactor=1.989990
=== RFdc Initialized - Running Multi-tile Sync div=2===

=== Run DAC Sync ===
INFO : DAC Multi-Tile-Sync completed successfully

=== Run ADC Sync ===
INFO : ADC Multi-Tile-Sync completed successfully
DAC NCO frequency of Tile#0 Block0 is set to 1500.000000,1
DAC NCO frequency of Tile#0 Block1 is set to 1500.000000,1
DAC NCO frequency of Tile#1 Block0 is set to 1500.000000,1
DAC NCO frequency of Tile#1 Block1 is set to 1500.000000,1
DAC NCO frequency of Tile#2 Block0 is set to 1500.000000,1
DAC NCO frequency of Tile#2 Block1 is set to 1500.000000,1
DAC NCO frequency of Tile#3 Block0 is set to 1500.000000,1
DAC NCO frequency of Tile#3 Block1 is set to 1500.000000,1
ADC NCO frequency of Tile#0 Block0 is set to -1500.000000,2,1
ADC NCO frequency of Tile#0 Block1 is set to -1500.000000,2,1
ADC NCO frequency of Tile#1 Block0 is set to -1500.000000,2,1
ADC NCO frequency of Tile#1 Block1 is set to -1500.000000,2,1
ADC NCO frequency of Tile#2 Block0 is set to -1500.000000,2,1
ADC NCO frequency of Tile#2 Block1 is set to -1500.000000,2,1
ADC NCO frequency of Tile#3 Block0 is set to -1500.000000,2,1
ADC NCO frequency of Tile#3 Block1 is set to -1500.000000,2,1


=== Multi-Tile Sync Report ===
DAC0: Latency(T1) =2680, Adjusted DelayOffset(T20) =  0
DAC1: Latency(T1) =2680, Adjusted DelayOffset(T20) =  0
DAC2: Latency(T1) =2680, Adjusted DelayOffset(T20) =  0
DAC3: Latency(T1) =2680, Adjusted DelayOffset(T20) =  0
ADC0: Latency(T1) =640, Adjusted DelayOffset(T20) =  0
ADC1: Latency(T1) =640, Adjusted DelayOffset(T20) =  0
ADC2: Latency(T1) =640, Adjusted DelayOffset(T20) =  0
ADC3: Latency(T1) =640, Adjusted DelayOffset(T20) =  0
RFdcDcp initial Successful
The SDR model is 840(4x4) rev V3!
The FPGA firmware version=0xaabbccdd
The fw lasted build at bb:cc:dd
The sw lasted build at 22:03:25 Sep  6 2023
Xadc Setup PASSED
MRF4 initial successful!
fpga temperature=56.493317
```

4) 通过回环射频缆将 TX 和 RX 回环，通过 chipscope 看到射频回环的结果



注：这里我们只观察了每一路的第一个采样点，RF Data Converter IP 是并行出来四个采样点。

附录 A-API 介绍

详细 API 介绍请参考官方文档：

https://docs.xilinx.com/r/en-US/pg269-rf-data-converter

此处只对常用的 API 进行介绍。

配置 RFDC 工作状态

XRFdc_SetMixerSettings

功能：传递 Mixer/NCO 设置用于更新相应的块级寄存器。使用新值更新驱动程序结构体。

函数原型：

```
u32 XRFdc_SetMixerSettings(XRFdc *InstancePtr, u32 Type, u32 Tile_Id, u32 Block_Id, XRFdc_Mixer_Settings *MixerSettingsPtr);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Type	RF-ADC 或 RF-DAC	RF-ADC:0 ; RF-DAC:1
Tile_Id	RF-ADC/RF-DAC tile 编号	0~3
Block_Id	块内的 RF-ADC/RF-DAC 块编号	0~3
*MixerSettingsPtr	指向传递混频器/NCO 设置的 XRFdc_Mixer_Settings 结构的指针	

XRFdc_SetQMCSettings

功能：QMC 设置被填充到相应的寄存器中。使用新值更新程序结构体。

函数原型：

```
u32 XRFdc_SetQMCSettings(XRFdc *InstancePtr, u32 Type, u32 Tile_Id, u32 Block_Id, XRFdc_QMC_Settings *QMCSettingsPtr);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Type	RF-ADC 或 RF-DAC	RF-ADC:0 ; RF-DAC:1
Tile_Id	RF-ADC/RF-DAC tile 编号	0~3
Block_Id	块内的 RF-ADC/RF-DAC 块编号	0~3
*QMCSettngs	指向用于更新相应寄存器的 XRFdc_QMC_Settings 结构的指针	

XRFdc_SetFabClkOutDiv

功能：使用此 API 设置 PL 时钟输出的分频器。

函数原型：

```
u32 XRFdc_SetFabClkOutDiv(XRFdc *InstancePtr, u32 Type, u32 Tile_Id, u16 FabClkDiv);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Type	RF-ADC 或 RF-DAC	RF-ADC:0 ; RF-DAC:1
Tile_Id	RF-ADC/RF-DAC tile 编号	0~3
Block_Id	块内的 RF-ADC/RF-DAC 块编号	0~3
FabClkDiv	为 tile 设置的 PL 时钟分频器	

时钟分频器有效值：

Macro	Description
XRFDC_FAB_CLK_DIV1	Divided by 1
XRFDC_FAB_CLK_DIV2	Divided by 2
XRFDC_FAB_CLK_DIV4	Divided by 4
XRFDC_FAB_CLK_DIV8	Divided by 8
XRFDC_FAB_CLK_DIV16	Divided by 16

配置 ADC 相关工作状态

XRFdc_SetDecimationFactor

功能：此 API 函数为请求的 RF-ADC 设置抽取因子，并根据抽取因子更新 FIFO 写入宽度。

函数原型：

```
u32 XRFdc_SetDecimationFactor(XRFdc *InstancePtr, u32 Tile_Id, u32 Block_Id,
u32 DecimationFactor);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Tile_Id	RF-ADC tile 编号	0~3
Block_Id	块内的 RF-ADC 块编号	0~3
DecimationFactor	RF-ADC 设置的抽取因子	

有效的抽取因子：

Macro	Description
XRFDC_INTERP_DECIM_OFF	Interpolation is OFF
XRFDC_INTERP_DECIM_1X	1X Decimation factor
XRFDC_INTERP_DECIM_2X	2X Decimation factor
XRFDC_INTERP_DECIM_3X	3X Decimation factor
XRFDC_INTERP_DECIM_4X	4X Decimation factor
XRFDC_INTERP_DECIM_5X	5X Decimation factor
XRFDC_INTERP_DECIM_6X	6X Decimation factor
XRFDC_INTERP_DECIM_8X	8X Decimation factor
XRFDC_INTERP_DECIM_10X	10X Decimation factor
XRFDC_INTERP_DECIM_12X	12X Decimation factor
XRFDC_INTERP_DECIM_16X	16X Decimation factor
XRFDC_INTERP_DECIM_20X	20X Decimation factor

XRFDC_INTERP_DECIM_24X	24X Decimation factor
XRFDC_INTERP_DECIM_40X	40X Decimation factor

XRFdc_SetFabRdVldWords

功能：此 API 函数通过写入相应的寄存器来设置所请求的 RF-ADC 的读取 PL 数据率。

函数原型：

```
u32 XRFdc_SetFabRdVldWords(XRFdc *InstancePtr, u32 Tile_Id, u32 Block_Id, u32 FabricRdVldWords);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Tile_Id	RF-ADC tile 编号	0~3
Block_Id	块内的 RF-ADC 块编号	0~3
FabricRdVldWords	RF-ADC 设置的 PL 数据速率	

XRDdc_SetDSA

功能：设置 DSA 值，更新相应的块级寄存器。

函数原型：

```
u32 XRFdc_SetDSA(XRFdc *InstancePtr, u32 Tile_Id, u32 Block_Id, XRFdc_DSA_Settings *SettingsPtr);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Tile_Id	RF-ADC tile 编号	0~3
Block_Id	块内的 RF-ADC 块编号	0~3
*SettingsPtr	指向 XRFdc_DSA_Settings 的指针，用于传递 DSA 设置。	

配置 DAC 相关工作状态

XRFdc_SetInterpolationFactor

功能：此 API 函数为请求的 RF-DAC 设置插值因子，并根据插值因子更新 FIFO 的读取宽度。

函数原型：

```
u32 XRFdc_SetInterpolationFactor(XRFdc *InstancePtr, u32 Tile_Id, u32 Block_Id, u32 InterpolationFactor);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Tile_Id	RF-DAC tile 编号	0~3
Block_Id	块内的 RF-DAC 块编号	0~3
InterpolationFactor	RF-DAC 设置的插值因子	

插值因子有效值：

Macro	Description
XRFDC_INTERP_DECIM_OFF	Interpolation is OFF
XRFDC_INTERP_DECIM_1X	1X interpolation factor
XRFDC_INTERP_DECIM_2X	2X interpolation factor
XRFDC_INTERP_DECIM_3X	3X interpolation factor
XRFDC_INTERP_DECIM_4X	4X interpolation factor
XRFDC_INTERP_DECIM_5X	5X interpolation factor
XRFDC_INTERP_DECIM_6X	6X interpolation factor
XRFDC_INTERP_DECIM_8X	8X interpolation factor
XRFDC_INTERP_DECIM_10X	10X interpolation factor
XRFDC_INTERP_DECIM_12X	12X interpolation factor
XRFDC_INTERP_DECIM_16X	16X interpolation factor
XRFDC_INTERP_DECIM_20X	20X interpolation factor
XRFDC_INTERP_DECIM_24X	24X interpolation factor

XRFDC_INTERP_DECIM_40X	40X interpolation factor

XRFdc_SetFabWrVldWords

功能：此 API 函数通过写入相应的寄存器来设置请求的 RF-DAC 的写入结构数据速率。

函数原型：

```
u32 XRFdc_SetFabWrVldWords(XRFdc *InstancePtr, u32 Tile_Id, u32 Block_Id, u32 FabricWrVldWords);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Tile_Id	RF-DAC tile 编号	0~3
Block_Id	块内的 RF-DAC 块编号	0~3
*FabricDataRatePtr	RF-DAC 设置的结构数据速率	

XRFdc_SetDACVOP

功能：VOP uA 电流用于更新相应的块级寄存器，动态调整信号强度。

函数原型：

```
u32 XRFdc_SetDACVOP(XRFdc *InstancePtr, u32 Tile_Id, u32 Block_Id, u32 uACurrent);
```

参数	参数说明	取值
*InstancePtr	指向驱动程序实例的指针	
Tile_Id	RF-DAC tile 编号	0~3
Block_Id	块内的 RF-DAC 块编号	0~3
uACurrent	电流，单位μA	2250 μA~40500 μA

注：对于 ES1 硅，电流范围是 6425 μA至 32000 μA（数值将被四舍五入到最近的增量）；对于生产性硅，范围是 2250 μA至 40500 μA（数据将被四舍五入到最近

的增量）。